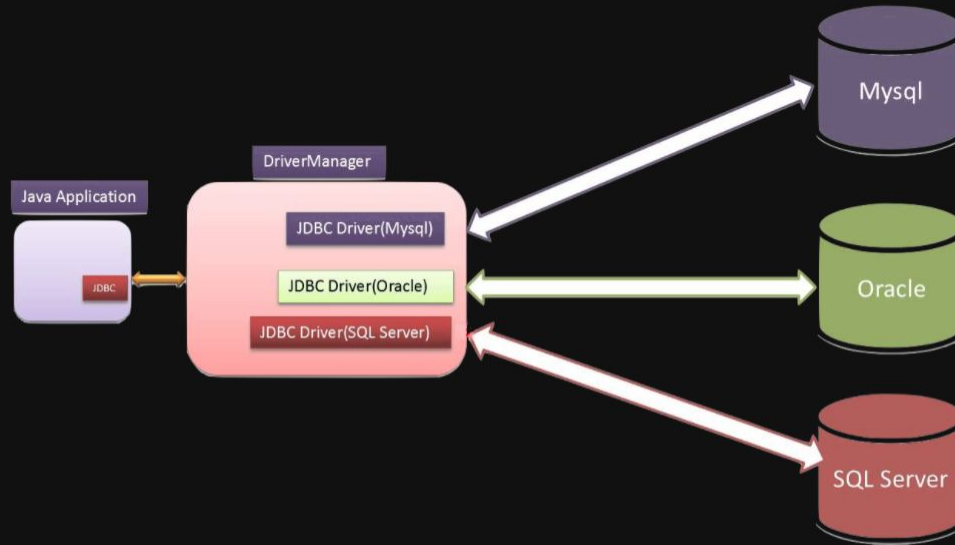


Java Database Connectivity

Created by <Academia de Código_>

JDBC

Java Database Connectivity is a standard API for Java programs to access database management systems



Functionality of JDBC

With JDBC we can:

- Connect to a data source, like a database
- Perform queries and update statements on data
- Retrieve and process the results received

Connection

- DriverManager Connects an application to a data source, which is specified by a database URL;
- Database URL Contains information such as where to search for the database, the name of the database to connect to, and configuration properties.

Database URL

`jdbc:mysql://[host][,failoverhost...][:port]/[database][?propertyName1][=propertyValue1]...`

- host:port is the hostname and port number of the computer hosting your database. If not specified, the default values of host and port are 127.0.0.1 and 3306, respectively;
- database is the name of the database to connect to;
- failover is the name of a standby database (MySQL Connector/J supports failover);
- propertyName=propertyValue represents an optional, ampersand-separated list of properties.

Connecting to the database

```
public class ConnectionManager {  
  
    private Connection connection = null;  
  
    public Connection getConnection() {  
  
        try {  
            if (connection == null) {  
                connection = DriverManager.getConnection(dbUrl, user, pass);  
            }  
        } catch (SQLException ex) {  
            System.out.println("Failure to connect to database : " + ex.getMessage());  
        }  
        return connection;  
    }  
  
    public void close() {  
        try {  
            if (connection != null) {  
                connection.close();  
            }  
        } catch (SQLException ex) {  
            System.out.println("Failure to close database connections: " + ex.getMessage());  
        }  
    }  
}
```

Statements

- A Statement is an interface that represents a SQL statement;
- You execute Statement objects, and they generate ResultSet objects, which represent a database result set.

Queries

- execute: use this method if the query could return one or more ResultSet objects (returns true if the first object is a ResultSet; get the ResultSet by calling Statement.getResultSet);
- executeQuery: returns one ResultSet object;
- executeUpdate: returns the number of rows affected by the SQL statement (use with INSERT, DELETE or UPDATE statements).

Creating and executing statements

```
public class JdbcUserService implements UserService {  
  
    private Connection dbConnection;  
  
    @Override  
    public int count() {  
  
        int result = 0;  
  
        // create a new statement  
        Statement statement = dbConnection.createStatement();  
  
        // create a query  
        String query = "SELECT COUNT(*) FROM user";  
  
        // execute the query  
        ResultSet resultSet = statement.executeQuery(query);  
  
        // get the results  
        if (resultSet.next()) {  
            result = resultSet.getInt(1);  
        }  
  
        return result;  
    }  
}
```

ResultSet Object

- We access the data in a ResultSet using a cursor that points to one row of data in the object
- To move the cursor to the next row, we use the method next from the ResultSet class, which returns false when no more rows are available
- Getters are available on the ResultSet class to retrieve data from the current row

Processing ResultSet Objects

```
@Override
public User findByName(String username) {

    // ... connection and statements....

    // execute the query
    ResultSet resultSet = statement.executeQuery(query);

    // user exists
    if(resultSet.next()) {

        String usernameValue = resultSet.getString("username");
        String passwordValue = resultSet.getString("password");
        String emailValue = resultSet.getString("email");

        user = new User(usernameValue, passwordValue, emailValue);
    }

    // ....
}
```

Closing the connection

It is important to close the connection and statement when no longer needed to release the used resources.

```
@Override
public User findByName(String username) {

    try {

        ...

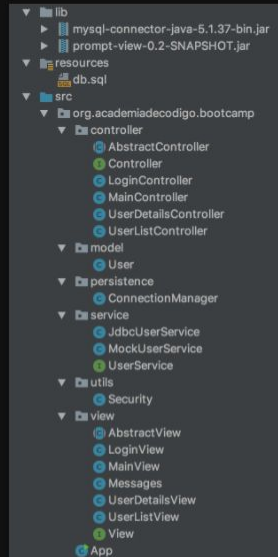
        if(statement != null) {
            statement.close();
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

}
```

{ Exercise }

Prompt-View Login with MVC, Service Layer and JDBC



Statement Types

- Statement: Used to implement SQL statements;
- PreparedStatement: Used for precompiling SQL statements that might contain input parameters;
- CallableStatement: Used to execute stored procedures that may contain both input and output parameters.

Prepared Statements

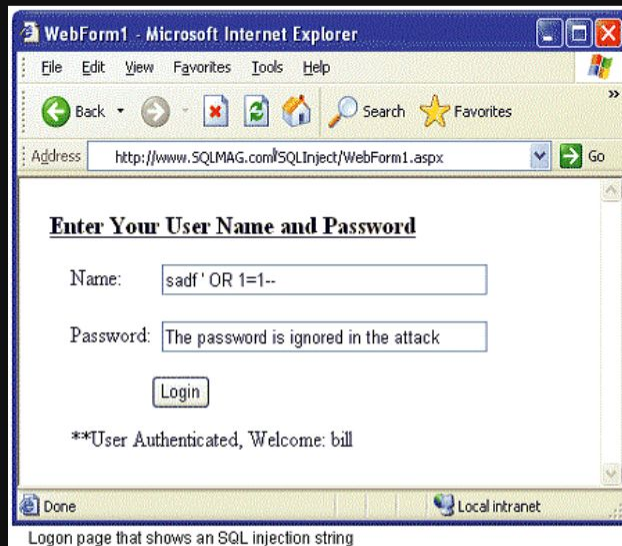
- It needs a SQL statement when it is created;
- The SQL statement is sent to the DBMS where it is compiled;
- Reduces execution time when executing a statement multiple times.
- Prevents SQL injection attacks

Creating and executing prepared statements

```
public User getUser(int id, String name) {  
    ...  
  
    // create a query  
    String query = "SELECT * FROM table WHERE id=? AND name=?";  
    PreparedStatement statement = dbConnection.prepareStatement(query);  
  
    // set values for the placeholders  
    statement.setInt(1, id);  
    statement.setString(2, name);  
  
    // execute the query  
    ResultSet resultSet = statement.executeQuery();  
  
    ...  
}
```

SQL Injection

Insertion of malicious SQL query via the input data from the client to the application



{ Hands On }

Demonstrate that we can login without knowing the password using SQL injection

{ Exercise }

Fix SQL injection vulnerabilities in the JdbcUserService